AWS Certified Solutions Architect Associate (SAA-C03) Study Notes

Based on "More Solution Architectures" Section

1. Event Processing in AWS

Decoupling applications using SQS, SNS, and Lambda.

SQS & Lambda

- **Pattern:** SQS Queue → Lambda Function.
- **Mechanism:** Lambda **polls** the SQS queue.
- **Behavior:** If processing fails, the message is returned to the queue and retried. Eventually, it can go to a Dead Letter Queue (DLQ).

SNS & Lambda

- **Pattern:** SNS Topic → Lambda Function.
- **Mechanism: Asynchronous** push. SNS pushes the message to Lambda.
- **Behavior:** If Lambda fails, SNS can retry (configurable). A DLQ can be attached to Lambda for failed events.

Fan Out Pattern

- **Problem:** You want to send one message to multiple independent processing queues.
- **Solution:** Push once to SNS → SQS Queues subscribe to the SNS Topic.
- **Architecture:** SDK → SNS Topic → Queue A (Process A) & Queue B (Process B).

S3 Event Notifications

- **Triggers:** `S3:ObjectCreated`, `S3:ObjectRemoved`, etc.
- **Features:** Filter by object name (e.g., `*.jpg`).
- **Destinations:** SNS, SQS, Lambda.
- **Latency:** Typically seconds, but can take minutes.
- **Pattern:** Upload Image → S3 → SQS → Lambda (to generate thumbnail). *Using SQS ensures messages aren't lost if Lambda is overwhelmed.*

S3 & EventBridge

- **Evolution:** S3 can send events to **Amazon EventBridge**.
- **Capabilities:**
  - Advanced filtering (metadata, object size).

- Over 18 AWS service destinations (Step Functions, Kinesis, etc.).
- Archive/Replay events capabilities.

Intercepting API Calls (EventBridge)

- **Use Case:** React to infrastructure changes (e.g., a DynamoDB table deletion).
- **Flow:** User calls `DeleteTable` → CloudTrail logs API call → EventBridge Rule matches event → SNS (Alert).

API Gateway & Kinesis

- **Pattern:** High throughput data ingestion.
- **Flow:** Client → API Gateway → Kinesis Data Streams → Kinesis Data Firehose → Amazon S3 (store as `.json`).

2. Caching Strategies in AWS

Caching reduces latency and offloads compute/database resources.

Layer	Technology	Details
Edge	**CloudFront**	Caches content close to the user. TTL, Geographic routing. Best for static content.
Gateway	**API Gateway**	Caches API responses. Reduces load on backend Lambda/EC2.
Compute	**EC2 / Lambda**	Local caching (if applicable/stateful) or managing logic.
Database	**ElastiCache / DAX**	**Redis/Memcached:** Cache for RDS/DB. **DAX:** In-memory cache specifically for DynamoDB.

3. Blocking an IP Address in AWS

Strategies depend on where the traffic is entering.

1. VPC / Subnet Level (NACL)

- **Component:** Network ACL (NACL).
- **Mechanism:** NACLs allow **Deny** rules.
- **Action:** Create a Deny rule for the specific IP address as the first rule (low number).
- **Pros:** Efficient, blocks traffic before it hits the instance.

2. EC2 Instance Level (Security Group)

- **Component:** Security Group (SG).
- **Mechanism:** SG *only* supports **Allow** rules.

- **Limitation:** You **cannot** explicitly block an IP. You can only "block" it by *not* having an allow rule for it (which is hard if you need to allow `0.0.0.0/0` for a public web server).
- **Verdict:** NOT helpful for blocking specific IPs while allowing the rest of the world.

3. Application Load Balancer (ALB) Level

- **Component: AWS WAF** (Web Application Firewall) or NACL (at ALB subnet).
- **WAF:** Attach a Web ACL to the ALB. Use **IP Address Filtering** rules to block malicious IPs.
- **NACL:** Can block IPs at the subnet where ALB resides.

4. CloudFront Level (Edge)

- **Scenario:** Client $\rightarrow$ CloudFront $\rightarrow$ ALB $\rightarrow$ EC2.
- **Issue:** The ALB sees CloudFront's IP, not the Client's IP. Therefore, **NACLs at the ALB subnet are NOT helpful** for blocking the Client IP.
- **Solution:**
  - **AWS WAF:** Attach WAF to **CloudFront**. Block IPs at the edge.
  - **CloudFront Geo Restriction:** Block entire countries.

4. High Performance Computing (HPC) on AWS

Running complex, compute-intensive workloads.

Data Management & Transfer

- **AWS Direct Connect:** Move GB/s of data over private secure networks.
- **Snowball / Snowmobile:** Move PB of data physically.
- **AWS DataSync:** Automate data transfer between on-prem and S3/EFS/FSx.

Compute & Networking

- **EC2 Instances:** Use CPU-optimized or GPU-optimized instances.
- **Spot Fleets:** Cost savings for stateless workloads.
- **Placement Groups:** Use **Cluster** placement groups for low-latency, 10Gbps+ networking between instances (Same Rack/AZ).
- **Enhanced Networking (ENA):** Up to 100 Gbps.
- **Elastic Fabric Adapter (EFA):**
  - Bypasses Linux OS for low-latency transport.
  - Great for **MPI** (Message Passing Interface) and tightly coupled workloads.

Storage

- **Instance Store:** Highest performance (millions of IOPS), low latency, but ephemeral (data lost on stop).

- **EBS (io2 Block Express):** Up to 256,000 IOPS.
- **Amazon FSx for Lustre:** HPC-optimized distributed file system backed by S3.

Automation

- **AWS Batch:** Multi-node parallel jobs, schedules instances.
- **AWS ParallelCluster:** Open-source tool to deploy HPC clusters (automates VPC, Subnet, EFA setup).

5. EC2 Instance High Availability

Methods to ensure an EC2 instance (or its service) remains available or recovers quickly.

Method 1: CloudWatch Events + Lambda (Passive)

- **Architecture:**
  - Active Instance: Has an **Elastic IP**.
  - Standby Instance: Stopped.
- **Flow:** CloudWatch Alarm (monitor) $\rightarrow$ Triggers Lambda $\rightarrow$ Lambda starts Standby Instance & moves the Elastic IP to it.

Method 2: Auto Scaling Group (ASG) + User Data

- **Architecture:**
  - ASG Settings: Min 1, Max 1, Desired 1. Spans $\geq$ 2 AZs.
  - **Elastic IP** is unattached initially or detached from failed instance.
- **Flow:**
  - Instance fails. ASG terminates it.
  - ASG launches new instance in a new AZ.
  - **EC2 User Data** script runs on boot $\rightarrow$ API call to **attach the specific Elastic IP** to itself.
  - *Requirement:* EC2 Instance Role must allow API calls for IP attachment.

Method 3: ASG + EBS Snapshots (Stateful Recovery)

- **Scenario:** You need to preserve data (state) upon recovery.
- **Flow:**
  - **On Terminate (Lifecycle Hook):** Take EBS Snapshot of the volume.
  - **On Launch (Lifecycle Hook):** New instance searches for latest snapshot $\rightarrow$ Creates Volume from Snapshot $\rightarrow$ Attaches Volume.